- Use parallel computing and GPU acceleration for faster model training.
- Optimize memory usage for handling large-scale datasets.
- Employ caching mechanisms to store frequently accessed recommendations.

Outcome:

The recommendation system will enhance user engagement by providing personalized product suggestions, leading to increased sales and improved customer satisfaction. It will also demonstrate the power of data-driven decision-making in the e-commerce industry.

Example Code:

```
import pandas as pd
from surprise import Dataset, Reader, SVD
from surprise.model_selection import train_test_split, accuracy # Step 1: Load Data
data = {
    'user_id': [1, 1, 2, 2, 3, 3],
    'item_id': [101, 102, 101, 103, 102, 104], 'rating': [5, 3, 4, 2, 1, 5]

                                                                                    }

df = pd.DataFrame(data)
reader = Reader(rating_scale=(1, 5))
data = Dataset.load_from_df(df[['user_id', 'item_id', 'rating']], reader) # Step 2: Train-Test Split
trainset, testset = train_test_split(data, test_size=0.2) # Step 3: Train Model
model = SVD()
model.fit(trainset)

# Step 4: Evaluate Model
predictions = model.test(testset)
accuracy.rmse(predictions)

# Step 5: Make Recommendations
def recommend(user_id, model, data, n=5): all_items = set(df['item_id'].unique()) rated_items =
set(df[df['user_id'] == user_id]['item_id']) unrated_items = all_items - rated_items recommendations
= [
        (item, model.predict(user_id, item).est) for item in unrated_items

                                                                                    ]

    recommendations.sort(key=lambda x: x[1], reverse=True) return recommendations[:n]
```